

LegalEase: AI-Powered Legal Document Generator

Team Name	Cipher
Team ID	SWTID-2026-6275
Team Leader	U. Harikrishnan
Team Members	N. Manikandan, B. Sri Harani Devi, K. Kesavan
Problem Statement	LegalEase: AI-Powered Legal Document Generator

Project Description:

LegalEase harnesses Generative AI to turn a few plain-language details into a complete, well-structured legal document. Many people, small businesses and students cannot afford a lawyer for every routine agreement, and writing one from scratch is slow and error-prone. LegalEase removes that barrier. The platform includes a Document Generator module, which drafts a full document (parties, clauses, obligations, termination, governing law and signature block) for the selected document type, a Plain-English Summary module, which explains the draft in a few simple lines so a non-lawyer can understand what they are signing, and a Risk Flag module, which lists clauses that deserve a second look before the document is signed.

Utilizing Groq's API with the LLaMA 3.3-70B Versatile model, LegalEase processes the user's inputs to deliver fast, consistent, professionally worded drafts. Built with Flask and deployed publicly via Ngrok, the platform offers a simple one-page experience: choose a document type, describe the parties and key terms, and receive a ready-to-review draft that can be copied or downloaded as a Word file. With secure API key management via .env, input validation, and a clear "not legal advice" disclaimer on every output, LegalEase empowers individuals, freelancers, startups and small businesses to prepare routine legal paperwork with confidence.

Scenarios

Scenario 1: A tenant and a landlord in Chennai need a rental agreement. The landlord selects Rental Agreement, enters the names, the monthly rent of ₹18,000, a security deposit of three months and an 11-month term, and selects the Formal style. LegalEase generates a complete agreement with rent, deposit, maintenance, notice period and termination clauses, a short plain-English summary, and flags such as "deposit refund timeline is not defined".

Scenario 2: A startup founder wants a freelancer to sign a Non-Disclosure Agreement (NDA) before sharing the product roadmap. The founder selects NDA with the Plain English style. LegalEase produces a clear confidentiality agreement that defines confidential information, the duration of the obligation, permitted disclosures and consequences of a breach, written in simple language that both sides can follow easily.

Scenario 3: A freelance designer needs a Service Agreement for a new client. The designer enters the scope of work, three payment milestones and a revision limit. LegalEase generates an agreement covering deliverables, payment schedule, intellectual-property ownership, late-payment terms and dispute resolution, and highlights that "ownership of source files after final payment" should be stated explicitly.

Technical Architecture:

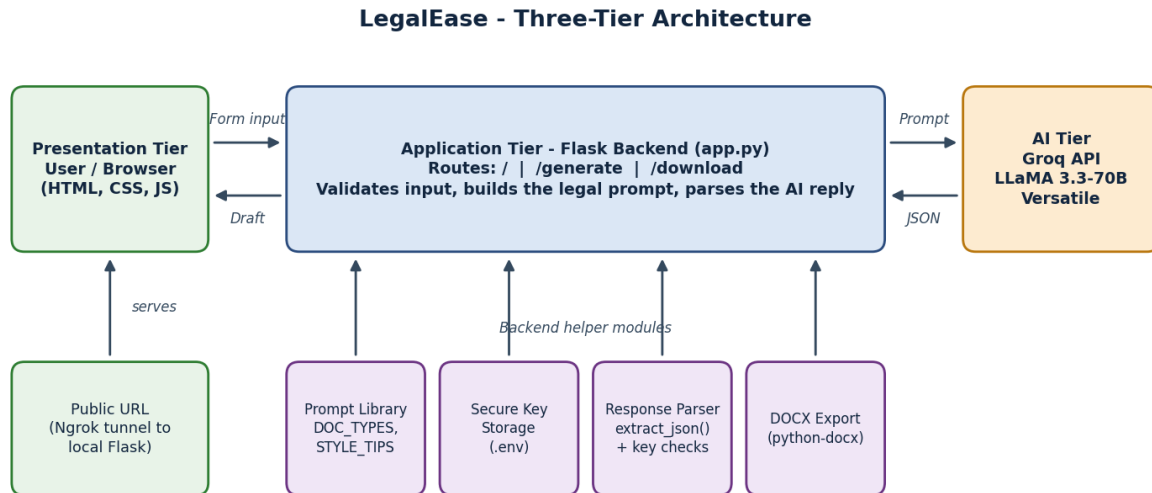


Figure 1: LegalEase technical architecture

Description: LegalEase utilizes a modular three-tier architecture to deliver rapid, AI-driven legal drafting. The presentation tier is a responsive single-page interface built with HTML, CSS and JavaScript, where the user selects the document type, jurisdiction, style and enters the key details. The application tier is a Flask backend that validates and sanitizes the input, builds a structured prompt from the prompt library (DOC_TYPES and STYLE_TIPS), calls the AI model, parses and verifies the JSON reply, and can export the final draft as a Word (.docx) file using python-docx. The AI tier uses the Groq API with the LLaMA 3.3-70B Versatile model to generate the document, a plain-English summary and a list of risk flags. The API key is stored in a .env file and is never exposed to the browser. The app runs locally on Flask and is exposed publicly through an Ngrok tunnel.

Database / ER Diagram: LegalEase does not use a database. Each request is stateless: user inputs are sent to the AI model, the result is returned to the browser, and nothing is stored on the server. This also protects user privacy because no agreement text is saved. Therefore an ER diagram is not applicable to this project.

Pre-requisites:

- **Flask Framework Knowledge:** [Flask Documentation](#)
- **Groq API Familiarity:** [Groq API Documentation](#)
- **HTML, CSS, and JavaScript Skills:** [W3Schools HTML/CSS/JavaScript Tutorials](#)
- **Python Programming Proficiency:** [Python Documentation](#)
- **python-docx for Word Export:** [python-docx Documentation](#)
- **Version Control with Git:** [Git Documentation](#)
- **Development Environment Setup:** [Flask Installation Guide](#)
- **Ngrok for Public Deployment:** [Ngrok Documentation](#)

Project Workflow:

Milestone 1: Model Selection and Architecture

- **Activity 1.1:** Generate a Groq API key and configure it securely in the project environment.
- **Activity 1.2:** Research and select the appropriate generative AI model from Groq for legal document drafting.
- **Activity 1.3:** Define the architecture of the application, detailing interactions between the frontend, backend, and AI integration.
- **Activity 1.4:** Set up the development environment, installing necessary libraries and dependencies for Flask, Groq and python-docx.

Milestone 2: Core Functionalities Development

- **Activity 2.1:** Develop the core functionalities: Document Generator, Plain-English Summary, and Risk Flags.
- **Activity 2.2:** Implement the Flask backend to manage routing and user input processing, ensuring smooth API interactions.

Milestone 3: App.py Development

- **Activity 3.1:** Write the main application logic in app.py, establishing routes for generation and download and integrating AI responses.

Milestone 4: Frontend Development

- **Activity 4.1:** Design and develop the user interface using HTML, CSS, and JavaScript, ensuring a responsive and intuitive layout.
- **Activity 4.2:** Create dynamic rendering of the generated document, summary and risk flags, with copy and download actions.

Milestone 5: Deployment

- **Activity 5.1:** Prepare the application for local deployment by configuring the server environment and ensuring all dependencies are correctly installed.
- **Activity 5.2:** Deploy the application publicly using Ngrok to expose the local Flask server over a secure public URL.

Milestone 6: Conclusion

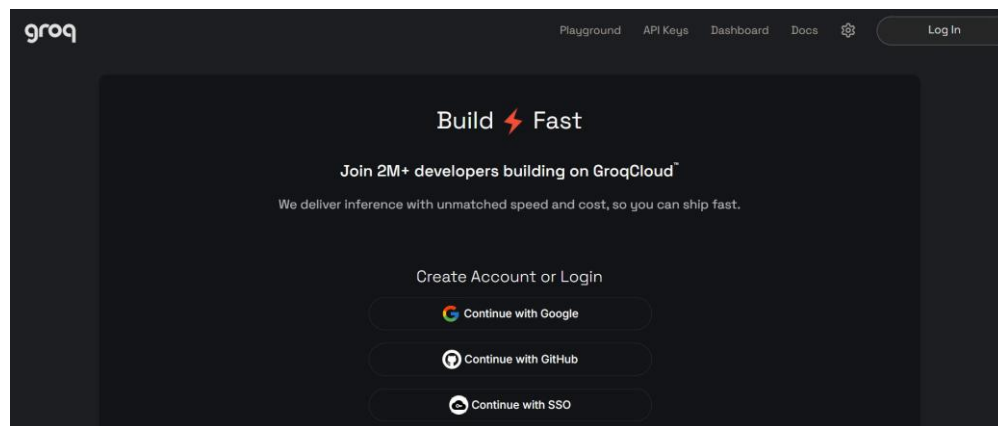
Milestone 1: Model Selection and Architecture

In this milestone, we focus on selecting the appropriate generative AI model from Groq for our legal document generation needs. This involves researching the capabilities and performance of the models Groq offers, ensuring that the chosen model aligns well with the application's objectives of drafting accurate, well-structured legal documents, summarizing them in plain English and flagging risky clauses.

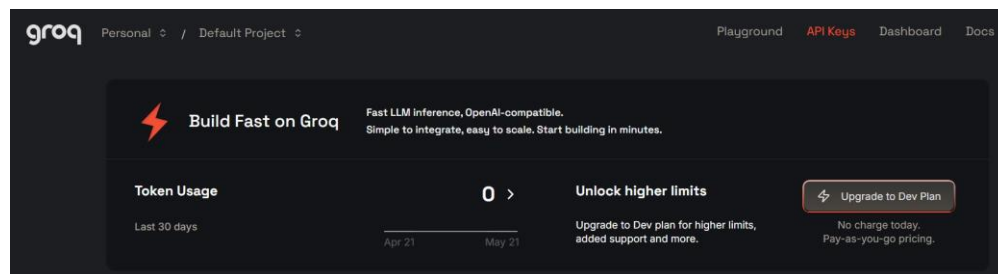
Activity 1.1: Generate a Groq API Key

Before the application can communicate with the Groq LLM, you need a valid Groq API key. Follow the steps below to create one and connect it securely to the project.

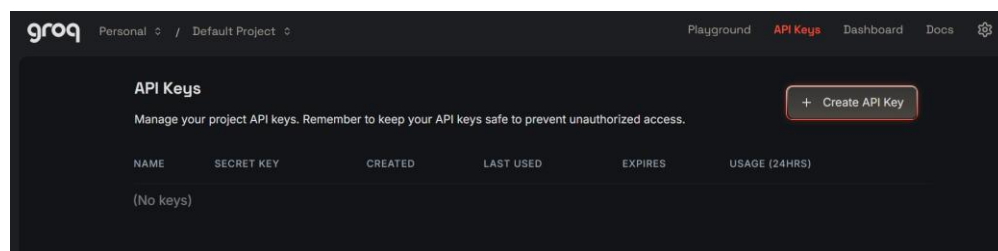
- **Step 1 — Create a Groq Account:** Visit <https://console.groq.com> and sign up for a free account using your Google account or email address.



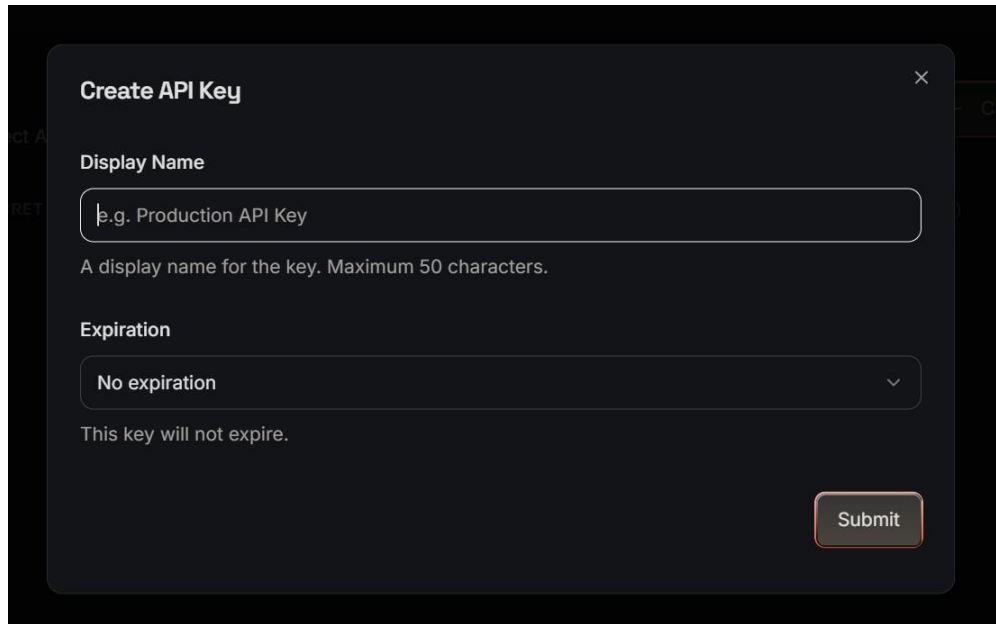
- **Step 2 — Navigate to API Keys:** After logging in, click on your profile icon in the top-right corner and select "API Keys" from the dropdown menu, or go directly to the [API Keys page](#).



- **Step 3 — Create a New API Key:** Click the "Create API Key" button. Give your key a descriptive name (for example, legalease-key) so it is easy to identify later.



- **Step 4 — Copy the API Key:** Give a "Display Name" ("Expiration" is optional) and click "Submit". Copy the key immediately and store it in a safe place; you will not be able to view it again after closing the dialog.



- **Step 5 — Add the Key to Your Project:** Create a .env file in the root of your project directory (if it does not already exist) and paste the key as shown below:

```
GROQ_API_KEY=your_copied_api_key_here
NGROK_AUTHTOKEN=your_ngrok_authtoken_here
```

- **Step 6 — Verify the Key is Loaded:** The app.py file loads this key automatically at startup using python-dotenv:

```
from dotenv import load_dotenv
load_dotenv()
client = Groq(api_key=os.environ.get("GROQ_API_KEY"))
```

- **Important — Never Commit Your API Key:** Add .env to your .gitignore file to prevent accidentally pushing the key to a public repository:

```
.env
venv/
myenv/
__pycache__/
```

Activity 1.2: Research and Select the Appropriate Generative AI Model

Here are the things that need to be researched to select the best model for the project:

- **Understand the Project Requirements:** Review the specific needs of LegalEase: long, structured outputs (a full agreement), strict JSON output, and consistent, formal legal wording across document types such as rental agreements, NDAs, employment contracts, service agreements and affidavits.
- **Explore Groq's Model Documentation:** Visit the Groq documentation to examine the available LLM models, including their functionalities, context windows, speed, and limitations.
- **Evaluate Model Performance:** Compare models based on instruction-following, accuracy of legal phrasing, ability to keep a long document consistent, and generation speed.
- **Select the Optimal Model:** Choose **llama-3.3-70b-versatile** for its balance of reasoning quality and speed. Use a low temperature of 0.3 so that clauses stay precise and repeatable rather than creative, and a max_tokens of 4096 so complete multi-page documents are not cut off.

Activity 1.3: Define the Architecture of the Application

- **Draft an Architectural Diagram:** Create a visual representation of the application architecture, including the frontend (HTML, CSS, JavaScript), the Flask backend, and the Groq API integration (see Figure 1).
- **Detail Frontend Functionality:** Outline how users interact with the application: selecting a document type, entering the jurisdiction, choosing a style (Formal or Plain English) and describing the parties and key terms.
- **Outline Backend Responsibilities:** Specify how the Flask backend handles incoming POST requests to /generate, validates and sanitizes user input, builds a structured prompt, communicates with the Groq API, and serves /download to export a .docx file.
- **Describe AI Integration Points:** Define how the backend builds prompts using the DOC_TYPES and STYLE_TIPS dictionaries, calls the Groq API, parses the JSON response via extract_json(), checks that the four required keys are present, and returns structured results to the frontend.

Activity 1.4: Set Up the Development Environment

- **Install Python and Pip:** Ensure Python 3.8+ is installed on your machine along with pip for dependency management.
- **Create a Virtual Environment:** Set up a virtual environment using venv to isolate project dependencies:

```
D:\projects>python -m venv myenv
D:\projects>myenv\Scripts\activate
(myenv) D:\projects>pip install -r requirements.txt
```

- **Install Flask:** Use pip to install Flask:

```
(myenv) D:\projects>pip install Flask==3.0.3
```

- **Install Groq SDK:** Install the Groq Python library to interact with the Groq API:

```
(myenv) D:\projects>pip install groq==0.11.0
```

- **Install python-dotenv and python-docx:** Install dotenv support for secure API key management and python-docx for Word export:

```
(myenv) D:\projects>pip install python-dotenv==1.0.1
(myenv) D:\projects>pip install python-docx==1.1.2
```

- **Configure the API Key:** Create a .env file in the project root and add your Groq API key:

```
GROQ_API_KEY=your_groq_api_key_here
```

- **Set Up Application Structure:** Create the project directory structure with folders for templates, static files (CSS, JS), tests, and the main application files:

```
legalease-ai/
|-- app.py
|-- run_public.py
|-- requirements.txt
|-- .env.example
|-- .gitignore
|-- README.md
|-- templates/
|   |-- index.html
|-- static/
|   |-- css/style.css
|   |-- js/main.js
|-- tests/
```

```
`-- test_app.py
```

- The requirements.txt file lists every dependency with its version:

```
Flask==3.0.3  
groq==0.11.0  
python-dotenv==1.0.1  
python-docx==1.1.2  
pyngrok  
pytest
```

Milestone 2: Core Functionalities Development

Activity 2.1: Develop Core Legal Drafting Features

Implement the three core features, the Document Generator, the Plain-English Summary and the Risk Flags, as one unified AI response driven by a single structured prompt. The model is asked to return strict JSON with the keys title, document, summary and risks.

- **Document Generator:** Drafts a complete document for the chosen type with a title, date, parties, recitals, numbered clauses, governing law for the selected jurisdiction, and a signature block. Supported types: Rental Agreement, Non-Disclosure Agreement (NDA), Employment Contract, Service Agreement and Affidavit. Details the user did not provide are left as clear blanks such as [____], never invented.
- **Plain-English Summary:** Returns a short summary (3 to 5 lines) that explains in simple words what each party must do, the key amounts and dates, and how the agreement can end.
- **Risk Flags:** Returns a list of up to five clauses or missing items that deserve attention (for example, a missing notice period, an undefined payment deadline or one-sided termination rights) so the user knows what to check or discuss with a lawyer.

Activity 2.2: Implement the Flask Backend

Define Routes in Flask:

- Set up routes in app.py for the home page, the generation endpoint (/generate) and the Word export endpoint (/download).

Process User Input:

- Create a form in index.html for users to select the document type, jurisdiction and style, and describe the parties and key terms.
- Use Flask's request.get_json() to retrieve JSON data submitted from the frontend via the Fetch API.
- Validate that doc_type is one of the supported types, and that details is non-empty and within the 3000-character limit before processing.

Integrate API Calls:

- Within the /generate route handler, call the build_prompt() function to construct a structured prompt combining the document-type instructions, the jurisdiction and the style tips.
- Submit the prompt to the Groq API using client.chat.completions.create() with the llama-3.3-70b-versatile model.
- Parse and validate the JSON response using extract_json(), which handles direct JSON, markdown code blocks and raw JSON object extraction as fallback strategies.

Milestone 3: App.py Development

Milestone 3 focuses on creating and refining the core logic of the app.py file, which serves as the backbone of the LegalEase application. In this milestone, you set up each feature through Flask routes, establish reliable prompt construction and API interaction, and write unit tests to ensure each part performs as expected.

Activity 3.1: Writing the Main Application Logic in app.py

Set Up the Application:

- Import the libraries, load the environment variables and create the Flask app and the Groq client:

```
import os, io, json, re
from flask import Flask, render_template, request, jsonify, send_file
from dotenv import load_dotenv
from groq import Groq
from docx import Document

load_dotenv()
app = Flask(__name__)
client = Groq(api_key=os.environ.get("GROQ_API_KEY"))
MODEL = "llama-3.3-70b-versatile"
```

Define Prompt Engineering Helpers:

- **DOC_TYPES** dictionary: maps each supported document type to the clauses the model must include.

```
DOC_TYPES = {
    "rental": ("Rental Agreement. Include parties, property, rent, deposit, term, "
              "maintenance, notice period, termination and governing law."),
    "nda": ("Non-Disclosure Agreement. Include definition of confidential "
            "information, obligations, exclusions, duration, remedies and law."),
    "employment": ("Employment Contract. Include role, salary, working hours, leave, "
                   "confidentiality, probation, notice and termination."),
    "service": ("Service Agreement. Include scope, deliverables, payment schedule, "
                "IP ownership, late payment, liability and dispute resolution."),
    "affidavit": ("Affidavit. Include deponent details, numbered statements of fact, "
                  "verification clause, place, date and signature."),
}
```

- **STYLE_TIPS** dictionary: maps the two writing styles to instructions for the model.

```
STYLE_TIPS = {
    "formal": "Use precise, formal legal wording with numbered clauses.",
    "plain": "Use simple, everyday English and short sentences, but keep every clause legally complete.",
}
```

- **build_prompt()**: assembles a structured prompt from the document type, jurisdiction, style and the user's details, and instructs the model to return strictly valid JSON.

```
def build_prompt(doc_type, details, jurisdiction, style):
    return f"""You are an experienced legal drafting assistant.
Draft a {DOC_TYPES[doc_type]}
Jurisdiction: {jurisdiction}
Style: {STYLE_TIPS.get(style, STYLE_TIPS["formal"])}
User details: {details}
Rules: do not invent facts; leave missing details as [____].
Return ONLY valid JSON with these keys:
    "title": short document title,
    "document": the full document text with numbered clauses,
    "summary": 3-5 line plain-English summary,
    "risks": list of up to 5 short risk flags."""
```

Parse the AI Response:

- **extract_json()**: a robust parser that first attempts `json.loads()`, then strips markdown code fences, and finally uses a regex to locate a raw JSON object, ensuring reliable parsing across model response variations.

```
def extract_json(text):
    try:
        return json.loads(text)
    except json.JSONDecodeError:
        pass
    m = re.search(r"```(?:json)?\s*([\s\S]*)```", text)
    if m:
        try:
            return json.loads(m.group(1).strip())
        except json.JSONDecodeError:
            pass
    m = re.search(r"\{([\s\S]*)\}", text)
    if m:
        try:
            return json.loads(m.group(0))
        except json.JSONDecodeError:
            pass
    raise ValueError("Could not extract valid JSON from response")
```

Define the Core Routes in app.py:

- **/**: home page route that renders the main `index.html` interface.

```
@app.route("/")
def index():
    return render_template("index.html")
```

- **/generate**: POST endpoint that accepts JSON input, validates it, invokes the AI and returns the structured result.

```
@app.route("/generate", methods=["POST"])
def generate():
    data = request.get_json() or {}
    doc_type = data.get("doc_type", "rental").lower()
    details = data.get("details", "").strip()
    jurisdiction = data.get("jurisdiction", "India").strip() or "India"
    style = data.get("style", "formal").lower()

    if doc_type not in DOC_TYPES:
        return jsonify({"error": "Unsupported document type."}), 400
    if not details:
        return jsonify({"error": "Please describe the parties and key terms."}), 400
    if len(details) > 3000:
        return jsonify({"error": "Input too long. Keep it under 3000 characters."}), 400

    try:
        completion = client.chat.completions.create(
            messages=[
                {"role": "system", "content": "You are a legal drafting assistant. Reply with valid JSON only."},
                {"role": "user", "content": build_prompt(doc_type, details, jurisdiction, style)},
            ],
            model=MODEL, temperature=0.3, max_tokens=4096,
        )
        result = extract_json(completion.choices[0].message.content)
        if not all(k in result for k in ["title", "document", "summary", "risks"]):
            raise ValueError("Invalid response structure from AI")
        return jsonify({"success": True, **result})
    except ValueError as e:
        return jsonify({"error": f"Failed to parse AI response: {e}"}), 500
    except Exception as e:
        return jsonify({"error": f"Generation failed: {e}"}), 500
```

- **/download** : POST endpoint that receives the generated text and returns it as a Word (.docx) file with the disclaimer appended.

```
@app.route("/download", methods=["POST"])
def download():
    data = request.get_json() or {}
    doc = Document()
    doc.add_heading(data.get("title", "Legal Document"), level=1)
    for line in data.get("document", "").split("\n"):
        if line.strip():
            doc.add_paragraph(line)
    doc.add_paragraph("Disclaimer: AI-generated draft. Not legal advice. "
                     "Please have it reviewed by a qualified lawyer.")
    buf = io.BytesIO(); doc.save(buf); buf.seek(0)
    return send_file(buf, as_attachment=True,
                    download_name="LegalEase_Document.docx")

if __name__ == "__main__":
    app.run(host="127.0.0.1", port=5050, debug=True)
```

Write Unit Tests (tests/test_app.py):

- Test `extract_json()` with plain JSON, fenced JSON and invalid text, and test that `/generate` returns HTTP 400 for an empty or oversized input. Run all tests with `pytest`:

```
(myenv) D:\projects> pytest tests/
```

- Route summary: `/` renders `index.html`; `/generate` (POST) accepts JSON and returns {success, title, document, summary, risks}; `/download` (POST) returns a .docx file.

Milestone 4: Frontend Development

Milestone 4 focuses on developing a clean, trustworthy and easy-to-use interface for LegalEase. This involves designing a responsive layout with HTML and CSS, and creating dynamic content rendering with vanilla JavaScript to display the AI-generated document, plain-English summary and risk flags based on user interactions.

Activity 4.1: Designing and Developing the User Interface

Set Up the Base HTML Structure:

- Develop the main HTML file (index.html) as the single-page interface, including a header with the LegalEase brand and tagline, an input section and a results section.
- Use semantic HTML elements to keep the structure organized and ensure the interface is accessible for all users.
- Integrate Font Awesome icons for document types and result headings, and Google Fonts (Inter) for clean typography.

Design a Responsive Layout Using CSS:

- Create a CSS stylesheet (static/css/style.css) with a calm, professional colour palette (navy and white) that suits a legal product.
- Use flexbox and grid layouts to organize the results into two columns: the full document on the left, and the summary and risk flags on the right.
- Add media queries to ensure the layout adapts across desktop, tablet and mobile screen sizes.

Create Separate UI Components for Each Output Type:

- **Document Viewer:** a scrollable page-style card that shows the generated document with clause numbering preserved, plus Copy and Download (.docx) buttons.
- **Summary Card:** a short card that shows the plain-English summary at the top of the results.
- **Risk Flag List:** a highlighted list of warning items with an alert icon, followed by a fixed disclaimer: "AI-generated draft. Not legal advice."

Activity 4.2: Creating Dynamic Content Rendering with JavaScript

Handle Form Submission Asynchronously:

- Attach a submit event listener to the form that prevents default submission and sends the data as a JSON POST request to /generate via the Fetch API.
- During loading, disable the Generate button and show an animated loader in its place.

Render Results Dynamically:

- Implement the renderResults(data) function in main.js, which dynamically creates and injects the document text, summary and risk flags into their containers.
- After rendering, smooth-scroll the viewport to the results using scrollToView(). The Download button posts the title and document text to /download and saves the returned .docx file.

Restore UI State After Generation:

- In the finally block of the async handler, re-enable the Generate button and restore its original text, regardless of success or failure. Errors from the server are shown in a friendly message box.

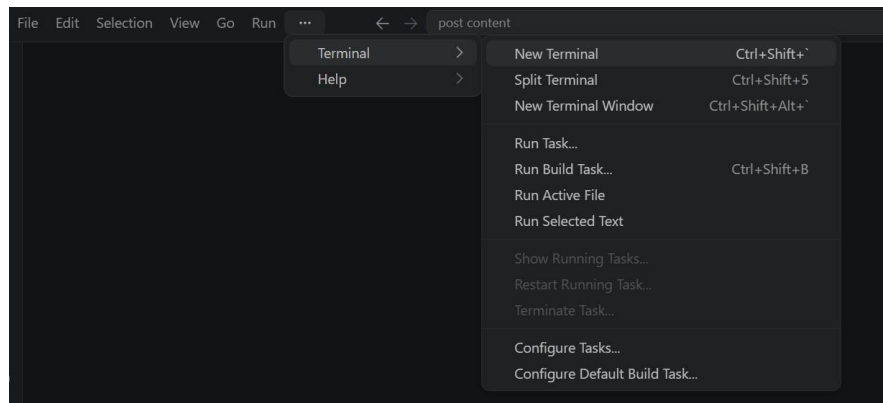
Milestone 5: Deployment

In Milestone 5, the focus is on deploying the LegalEase application first locally to verify correct operation, and then publicly via Ngrok to share the app over a secure, accessible URL. This involves configuring the server environment, managing dependencies and launching the app for real-world use.

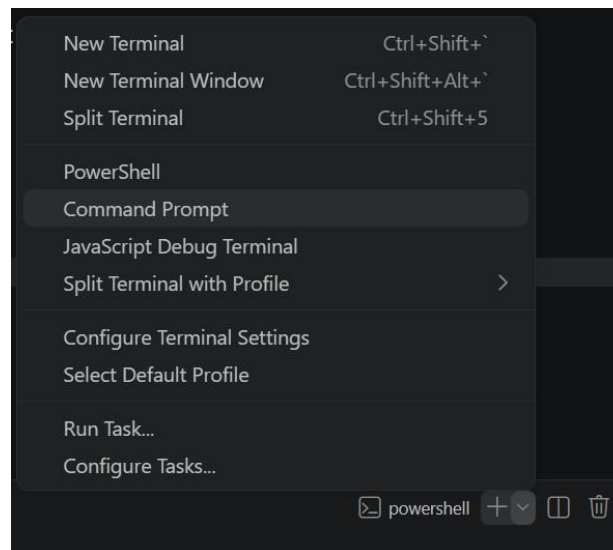
Activity 5.1: Preparing the Application for Local Deployment

Set Up a Virtual Environment:

- Open a New Terminal in your code editor.



- Then open "Command Prompt".



- Create and activate a virtual environment, then install all required dependencies from requirements.txt:

```
D:\projects>python -m venv myenv
D:\projects>myenv\Scripts\activate
(myenv) D:\projects>pip install -r requirements.txt
```

Configure Environment Variables:

- Create a .env file in the project root and add your Groq API key. The app loads this automatically using python-dotenv at startup:

```
GROQ_API_KEY=your_groq_api_key_here
```

Activity 5.2: Local Testing and Verification

Start the Flask Development Server:

- Run the application locally using:

```
(myenv) D:\projects>python app.py
```

- Once running, open a browser and navigate to "<http://127.0.0.1:5050>" to access the app.

[Insert screenshot: terminal showing the Flask server running on <http://127.0.0.1:5050>]

- Interact with the generator: test all five document types (Rental Agreement, NDA, Employment Contract, Service Agreement, Affidavit) and both styles (Formal, Plain English) to verify that the documents, summaries and risk flags are generated, parsed and rendered correctly, and that the Download button produces a valid .docx file.

Activity 5.3: Public Deployment via Ngrok

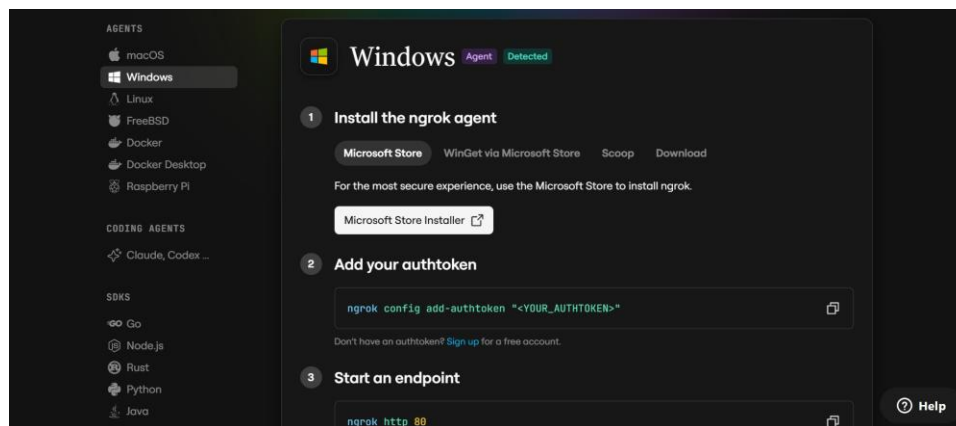
Ngrok creates a secure tunnel from a public URL to your local Flask server, making the app accessible from anywhere without a cloud hosting provider.

Install Ngrok and pyngrok:

- Install the pyngrok Python wrapper, which manages Ngrok programmatically within your Python project:

```
D:\projects>pip install pyngrok
```

- Download and install the Ngrok client from <https://ngrok.com/download>.
- Click on "Microsoft Store Installer".



Configure Your Ngrok Authtoken:

- Sign up at ngrok.com and copy your authtoken from the Ngrok dashboard.
- Add it to the .env file as NGROK_AUTHTOKEN. The run_public.py script reads it and sets it automatically:

```
import os, threading
from dotenv import load_dotenv
from pyngrok import ngrok
from app import app

load_dotenv()
ngrok.set_auth_token(os.environ.get("NGROK_AUTHTOKEN"))
tunnel = ngrok.connect(5050)
```

```
print("=" * 54)
print("YOUR PUBLIC WEBSITE URL IS LIVE!")
print("URL:", tunnel.public_url)
print("=" * 54)
app.run(host="127.0.0.1", port=5050, debug=False)
```

Run the Public Deployment Script:

- Instead of running `app.py` directly, launch `run_public.py` to start both the Ngrok tunnel and the Flask server:

```
D:\projects>python run_public.py
```

- The script opens an HTTP tunnel on port 5050 and prints the live public URL in the terminal:

```
=====
YOUR PUBLIC WEBSITE URL IS LIVE!
URL: https://xxxx-xx-xx-xx-xx.ngrok-free.app
=====
```

- Share this URL with anyone; they can access the LegalEase app directly from their browser without any installation.

Important Notes:

- **debug=False** is required in `run_public.py` to prevent Flask from spawning a reloader process, which would interfere with Ngrok's tunnel management.
- The public URL changes each time you restart `run_public.py` (on the free Ngrok plan). For a persistent URL, upgrade to a paid Ngrok plan and configure a reserved domain.
- Ngrok free-tier sessions expire after a few hours of inactivity. Restart `run_public.py` to get a new URL when this happens.

Exploring the Website's Web Pages:

Home / Generator Page:

[Insert screenshot: LegalEase home / generator page]

Description: The single-page interface of LegalEase serves as both the landing page and the generation tool. It features a clean navy-and-white design, a header showing the LegalEase brand and a tagline, and a short note that the output is a draft and not legal advice. The input form and the results panel sit together on one seamless page.

Input Section:

[Insert screenshot: input form with document type, jurisdiction, style and details]

Description: A card containing the main generation form. Users choose the document type (Rental Agreement, NDA, Employment Contract, Service Agreement or Affidavit), enter the jurisdiction, choose the style (Formal or Plain English), describe the parties and key terms in a text area (up to 3000 characters), and click the Generate Document button. While the AI processes the request, the button changes to an animated loader.

Results Section:

[Insert screenshot: results: document, summary and risk flags]

Description: Revealed dynamically after generation, this section shows the AI output in a two-column layout. The left column displays the full document with Copy and Download (.docx) buttons. The right column contains a Plain-English Summary card and a Risk Flags list, followed by the disclaimer. The viewport smooth-scrolls to this section as soon as the content is generated.

Conclusion:

LegalEase is a generative AI platform built with Flask that makes routine legal drafting fast, affordable and understandable. It uses Groq's high-speed inference API with the LLaMA 3.3-70B Versatile model to generate complete documents for rental agreements, NDAs, employment contracts, service agreements and affidavits, together with a plain-English summary and a list of risk flags. The project, which included model selection, backend and frontend development, Word export, and deployment via Ngrok, shows how targeted AI and a structured prompt can save time and reduce confusion for people who cannot easily consult a lawyer for every document. Because the output is an AI-generated draft, every result carries a clear disclaimer and a recommendation to have it reviewed by a qualified legal professional.

Looking ahead, LegalEase offers significant opportunities for expansion, such as supporting more document types and regional languages (including Tamil and Hindi), adding clause-by-clause editing and re-generation, uploading an existing contract for AI review, adding user authentication to save drafts and, with lawyer review, jurisdiction-specific template libraries. By continuously refining the prompts and improving accessibility, the platform can evolve from a focused drafting tool into a comprehensive legal-assistance companion for individuals and small businesses.